



PRACTICAL: 01

**Aim: To implement and find the time analysis of sorting algorithms.
Bubble sort, Selection sort, Insertion sort, Merge sort and Quick sort.**

A) Bubble Sort:

Algorithm:

Bubble Sort:-

Input:- Array B with n elements

Output:- Sorted array B in ascending order.

1. Start

2. Read n and array B.

3. Set $i = 0$

4. Repeat while $i < n - 1$:

 Set $j = 0$

 compare $B[j]$ and $B[j+1]$

 if $B[j] > B[j+1]$, swap them

 increment j until $j < n - i - 1$

 increment i

5. Display the sorted array B.

6. Stop

Program / Code:

```
#include<iostream>

using namespace std;

int main()
{
    int a[60], size, i, j, temp;

    cout<<"Enter the size of the array: ";

    cin>>size; cout<<"Enter the
    array elements: ";

    for(i=0; i<size; i++){
```



```
        cin>>a[i];
    }
    for(i=0; i<size-1; i++){
        for(j=0; j<size-i-1; j++){
            if(a[j]>a[j+1]){
                temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;
            }
        }
    }
    cout<<"The Elements are: ";
    for(i=0; i<size; i++){
        cout<<a[i];
    }
    return 0;
}
```

Output:

```
Enter the size of array: 5
Enter Array Elements: 2 8 3 0 1
Before Sorting
2 8 3 0 1
After Sorting
0 1 2 3 8

=== Code Execution Successful ===
```



Time Complexity:

```

for (int i = 0; i < n-1; i++) — outer loop
{
    for (int j = 0; j < n-1-i; j++) — inner loop
    {
        if (arr[j] > arr[j+1]) — 1
        {
            int temp = arr[j]; — 1
            arr[j] = arr[j+1]; — 1
            arr[j+1] = temp; — 1
        }
    }
}

```

outer loop: Runs from 0 to n-1
So it executes n-1 times

inner loop: Pass (i) iterations

0	n-1
1	n-2
...	...
n-2	1

$$(n-1) + (n-2) + \dots + 2 + 1 = 1 + 2 + \dots + (n-1)$$

$$= \frac{n(n-1)}{2}$$

$$= \frac{n^2 - n}{2}$$

$$T(n) = n-1 + \frac{n^2 - n}{2} + 1 + 1 + 1 + 1$$

$$= n + \frac{n^2 - n}{2} + 3$$

ignore all constants
by asymptotic
notation

$$T(n) = O(n^2) \rightarrow \text{worst case}$$

$$T(n) = O(n^2) \rightarrow \text{in Average case}$$

$$T(n) = O(n) \rightarrow \text{Best case}$$

B) Insertion Sort:

Algorithm:

Insertion Sort:-

Input:- Array A with n elements

Output:- Sorted array A in ascending order.

1. Start
2. Read n and array A
3. Set $i = 1$
4. Set $\text{Key} = A[i]$ and $j = i-1$
5. while $j \geq 0$ and $A[j] > \text{Key}$, shift $A[j]$ to $A[j+1]$
6. Insert Key at $A[j+1]$
7. Increment i and repeat steps 4-6 until $i < n$
8. Display sorted array A
9. Stop



Program / Code:

```
#include<iostream>

using namespace std;

int main(){

    int a[60], size, i, j, min, temp;

    cout<<"Enter the size of the array: ";

    cin>>size;

    cout<<"Enter the array elements: ";

    for(i=0; i<size; i++){

        cin>>a[i];

    }

    for(i=0; i<size-1; i++){

        min = i;

        for(j=i+1; j<size; j++){

            if(a[j] < a[min]){

                min = j;

            }

        }

        if(min != i){

            temp = a[i];

            a[i] = a[min];

            a[min] = temp;

        }

    }

    cout<<"The Elements are: ";

    for(i=0; i<size; i++){

        cout<<a[i]<<" ";

    }

}
```

```

    }

    return 0;

}

```

Output:

```

Enter the size of array: 5
Enter Array Elements: 3 9 0 5 1
Before Sorting
3 9 0 5 1
After Sorting
0 1 3 5 9

```

Time Complexity:

```

for (int i=1; i<n; i++)
{
    int Key = arr[i];
    int j = i-1;
    while (j >= 0 && arr[j] > Key)
    {
        arr[j+1] = arr[j];
        j--;
    }
    arr[j+1] = Key;
}

```

1. Best Case: Already sorted array
[10, 20, 30, 40, 50] → outer loop runs $n-1$ times
→ while checks only once for each element.
 $T(n) = O(n)$

2. Worst Case: Reverse Sorted array
[50, 40, 30, 20, 10]
each element must be compared with all previous elements and shifted.

Pass	Comparison
1	1
2	2
3	3
...	...
$n-1$	$n-1$

$$1+2+3+\dots+(n-1)$$

$$T(n) = \frac{n(n-1)}{2} = \frac{n^2-n}{2}$$

$$T(n) = O(n^2)$$

3. Average Case: Randomly arranged array
[30, 10, 50, 20, 40]
on average, each element is compared with about half of the previously sorted elements.

$$T(n) = \frac{1}{2} [1+2+3+\dots+(n-1)]$$

$$T(n) = \frac{n(n-1)}{4}$$

$$T(n) = O(n^2)$$



C) Selection Sort:

Algorithm:

Selection Sort

Input:- Array A with n elements

Output:- sorted array A in ascending order

1. Start
2. Read n and array A
3. Set $i = 0$
4. Repeat while $i < n - 1$:
 - Set $\text{min} = i$, $j = j + 1$
 - compare $A[j]$ with $A[\text{min}]$
 - if $A[j] < A[\text{min}]$, set $\text{min} = j$
 - increment j until $j < n$
 - swap $A[i]$ and $A[\text{min}]$
 - increment i
5. Display the sorted array A
6. stop

Program / Code:

```
#include <iostream>

using namespace std;

void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        int temp = arr[minIndex];
```

```
arr[minIndex] = arr[i];  
arr[i] = temp;  
}  
}  
int main() {  
    int n; cout << "Enter number of  
    elements: "; cin >> n; int arr[n];  
    cout << "Enter " << n << "  
    elements: "; for (int i = 0; i < n;  
    i++) {  
  
    cin >> arr[i];  
    }  
    selectionSort(arr, n);  
    cout << "Sorted array: ";  
    for (int i = 0; i < n; i++){  
        cout << arr[i] << " ";  
    }  
    return 0;  
}
```

Output:

```
Enter the size of array: 5  
Enter Array Elements: 3 9 0 5 1  
Before Sorting  
3 9 0 5 1  
After Sorting  
0 1 3 5 9  
  
=== Code Execution Successful ===
```



Time complexity:

selection sort

```

for (int i=0; i<n-1; i++) ————— n-1
{
    int minIndex = i; ————— 1
    for (int j=i+1; j<n; j++) —————  $\frac{n^2-n}{2}$ 
    {
        if (arr[j] < arr[minIndex]) — 1
        {
            minIndex = j; ————— 1
        }
    }
    int temp = arr[minIndex]; — 1
    arr[minIndex] = arr[i]; — 1
    arr[i] = temp; ————— 1
}

```

outer loop: The outer loop runs from i=0 to n-1
so it executes n-1 times.

inner loop:	Pass	Comparisons
	1	n-1
	2	n-2
	3	n-3
	⋮	⋮
	n-1	1

$(n-1) + (n-2) + \dots + 2 + 1$
 $T(n) = \frac{n(n-1)}{2} = \frac{n^2-n}{2}$

$$T(n) = n-1 + 1 + \frac{n^2-n}{2} + 1 + 1 + 1 + 1$$

$$= n + \frac{n^2-n}{2} + 4 = 2n + \frac{n^2-n}{2} + 8$$

$$= n^2 + n + 8$$

∴ (by asymptotic notation
avoid const)

$$= n^2 + n$$

$$T(n) = O(n^2) \text{ — highest power — worst case}$$

$$T(n) = O(n^2) \text{ — Best Case}$$

$$T(n) = O(n^2) \text{ — Average Case}$$

D) Merge Sort:

Algorithm:

Merge Sort:-

Input:- Array A with n elements.
Output:- Sorted Array A in ascending order.

1. Start
2. Read n and array A
3. Divide array A into two halves at the middle.
 $mid = (low + high) / 2$

4. Recursively apply Merge sort on left half.
5. Recursively apply Merge sort on Right half.
6. Merge the two sorted halves.

if ($LB[i] \leq UB[j]$)
 $A[k] = LB[i]$
 $i = i + 1$;

else

$A[k] = UB[j]$;
 $j = j + 1$;

7. Repeat step 6 until either LB or UB is fully placed into A.

8. Copy any remaining elements of LB or UB directly into A.

9. Repeat until entire array is merged & sorted.

10. Print sorted array.

11. End

Program / Code:

```
#include <iostream>
```

```
using namespace std;
```

```
void merge(int arr[], int left, int mid, int right) {
```

```
    int n1 = mid - left + 1;
```

```
    int n2 = right - mid;
```

```
    int L[n1], R[n2];
```

```
    for (int i = 0; i < n1; i++) {
```

```
        L[i] = arr[left + i];
```



```
    }  
    for (int j = 0; j < n2; j++) {  
        R[j] = arr[mid + 1 + j];  
    }  
    int i = 0;  
    int j = 0;  
    int k = left;  
    while (i < n1 && j < n2) {  
        if (L[i] <= R[j]) {  
            arr[k++] = L[i++];  
        } else {  
            arr[k++] = R[j++];  
        }  
    }  
    while (i < n1) {  
        arr[k++] = L[i++];  
    }  
    while (j < n2) {  
        arr[k++] = R[j++];  
    }  
}  
  
void mergeSort(int arr[], int left, int right) {  
    if (left < right) {  
        int mid = left + (right - left) / 2;  
        mergeSort(arr, left, mid);  
        mergeSort(arr, mid + 1, right);  
        merge(arr, left, mid, right);  
    }  
}
```



```
    }  
}  
  
int main() { int n; 11 cout << "Enter number of elements: ";  
  
    cin >> n;  
    int arr[n];  
    cout << "Enter elements: ";  
    for (int i = 0; i < n; i++) {  
        cin >> arr[i];  
    }  
    mergeSort(arr, 0, n - 1);  
    cout << "Sorted array using Merge Sort: ";  
    for (int i = 0; i < n; i++) {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
    return 0;  
}
```

Output:

```
Enter the size of array: 6  
Enter Array Elements: 2 8 0 1 5 3  
Before Sorting  
2 8 0 1 5 3  
After Sorting  
0 1 2 3 5 8  
  
=== Code Execution Successful ===
```



Time Complexity:

Merge Sort

the array is divided into two halves repeatedly

Number of divisions : $\log_2 n$ ($2T(\frac{n}{2})$)

merge's Each level merges all n elements
 $O(n)$

$$\text{Total time } T(n) = 2T(\frac{n}{2}) + n$$

$$T(n) = 2T(\frac{n}{2}) + n \rightarrow \textcircled{1}$$

$n \rightarrow \frac{n}{2}$ Put in $\textcircled{1}$

$$T(\frac{n}{2}) = 2T(\frac{n}{4}) + \frac{n}{2}$$

$$T(\frac{n}{2}) = 2T(\frac{n}{4}) + \frac{n}{2} \rightarrow \textcircled{2}$$

Sub $\textcircled{2}$ in $\textcircled{1}$

$$T(n) = 2(2T(\frac{n}{4}) + \frac{n}{2}) + n$$

$$= 2^2 T(\frac{n}{4}) + 2n \rightarrow \textcircled{3}$$

$n \rightarrow \frac{n}{4}$ Put in $\textcircled{3}$

$$T(\frac{n}{4}) = 2^2 T(\frac{n}{8}) + 2(\frac{n}{4}) \rightarrow \textcircled{4}$$

Sub $\textcircled{4}$ in $\textcircled{3}$

$$T(n) = 2^2 [2^2 T(\frac{n}{8}) + \frac{2n}{4}] + 2n$$

$$= 2^4 T(\frac{n}{8}) + 4n$$

$$T(n) = 2^k T(\frac{n}{2^k}) + kn$$

$$T(n) = O(n \log n) \rightarrow \text{Average Case}$$

$$T(n) = O(n \log n) \rightarrow \text{Worst Case}$$

$$T(n) = 2^k T(\frac{n}{2^k}) + kn$$

$$\frac{n}{2^k} = 1$$

$$n = 2^k$$

Apply log both sides

$$\log n = \log 2^k$$

$$k = \log n$$

$$T(n) = 2^{\log n} T(\frac{n}{2^{\log n}}) + \log n(n)$$

$$= n^{\log 2} + \frac{n}{2^{\log 2}} + \log n(n)$$

$$= n \log n + n$$

$$T(n) = O(n \log n)$$

Best Case



E) Quick Sort:

Algorithm:

Quick Sort

Input:- Array A with n elements.

Output:- Sorted Array A in ascending order.

1. Start
2. Read n and array A
3. Choose pivot as the first element of the current sub-array.
Pivot = A[low]
4. Move i from left and j from right, scanning toward each other.
while (A[i] ≤ Pivot and i < high)
i = i + 1
while (A[j] > Pivot and j > low)
j = j - 1
5. If i < j,
swap(A[i], A[j])
6. Repeat step 3, 4 until i & j cross (i ≥ j)
7. Place pivot in its correct sorted position
swap(A[low], A[j])
return j
8. Recursively apply Quick sort on left part.
9. Recursively apply Quick sort on right part.
10. Repeat until each sub-array has one or zero elements.
11. Print sorted array.
12. End

Program / Code:

```
#include <iostream>

using namespace std;

int partition(int arr[], int lb, int ub) {

    int pivot = arr[lb];

    int start = lb;
```



```
int end = ub;
while (start < end) {
    while (arr[start] <= pivot) {
        start++;
    }
    while (arr[end] > pivot) {
        end--;
    }
    if (start < end) {
        swap(arr[start], arr[end]);
    }
}
swap(arr[lb], arr[end]);
return end;
}

void quickSort(int arr[], int lb, int ub) {
    if (lb < ub) {
        int loc = partition(arr, lb,
            ub); quickSort(arr, lb, loc -
            1); quickSort(arr, loc + 1,
        } ub);
    }
}

int main() {
    int n;
    cout << "Enter size of array: ";
    cin >> n;
    int arr[n];
```

```
cout << "Enter " << n << " elements: ";  
for (int i = 0; i < n; i++) {  
    cin >> arr[i];  
}  
quickSort(arr, 0, n - 1);  
cout << "Sorted array using Quick Sort: ";  
for (int i = 0; i < n; i++) {  
    cout << arr[i] << " ";  
}  
return 0;  
}
```

Output:

```
Enter the size of array: 7  
Enter Array Elements: 9 0 3 7 1 5 6 2  
Before Sorting  
9 0 3 7 1 5 6  
After Sorting  
0 1 3 5 6 7 9  
  
=== Code Execution Successful ===
```



Time Complexity:

Quick sort

- The pivot divides the array into two equal halves
→ $2T(n/2)$

- For sorting we use n elements at last
 $O(n)$

$$T(n) = 2T(n/2) + n \quad \text{--- (1)}$$

$$n \rightarrow n/2 \text{ Put in (1)}$$

$$T(n/2) = 2T(n/4) + \frac{n}{2} \quad \text{--- (2)}$$

$$\text{Sub (2) in (1)}$$

$$T(n) = 2(2T(n/4) + \frac{n}{2}) + n$$

$$= 2^2 T(\frac{n}{4}) + \frac{2n}{2} + n$$

$$T(n) = 2^2 T(\frac{n}{2^2}) + 2n \quad \text{--- (3)}$$

$$n \rightarrow n/4 \text{ Put in (3)}$$

~~$$T(n) = 2^3 T(\frac{n}{8}) + 3n$$~~

$$T(\frac{n}{4}) = 2^2 T(\frac{n}{2^4}) + 2(\frac{n}{4}) \quad \text{--- (4)}$$

$$\text{Sub (4) in (3)}$$

$$T(n) = 2^2 [2^2 T(\frac{n}{2^4}) + \frac{2n}{4}] + 2n$$

$$= 2^4 T(\frac{n}{2^4}) + \frac{2^3 n}{4} + 2n$$

$$= 2^4 T(\frac{n}{2^4}) + 4n$$

$$T(n) = 2^k T(\frac{n}{2^k}) + kn$$

$$\frac{n}{2^k} = 1, n = 2^k$$

apply log on b.s

$$\log n = \log 2^k$$

$$k = \log n$$

$$T(n) = 2^{\log n} T(\frac{n}{2^{\log n}}) + \log n(n)$$

$$= n \log n + n$$

$$T(n) = O(n \log n) \rightarrow \text{Best Case}$$

$$T(n) = O(n \log n) \rightarrow \text{Average Case}$$

$$T(n) = O(n^2) \rightarrow \text{Worst Case}$$